Rapport de TD7: Initiation à l'algorithmique

Derrez Lorenzo

October 20, 2026

Exercice 1:

Question 1;

- Première séquence: $(0, 1, 3)$
- Deuxième séquence: $(0, 1, 7)$

  On retrouve bien 1,2, et 1.

Question 2;

Soit B un mot de longueur $|B| = n$.
Pour chaque indice on a 2 choix:

- on le garde;
- on ne le garde pas.

Puis on fait cela pour les n éléments $\implies 2 \times \ldots \times 2$ (n fois) $= 2^n$

Question 3;

Par rapport à l'exemple 1;

- $maxLG(B, C) = 5$
- un plus long sous-mot commun de B et C est: 11322

Question 4;

On définit la caractérisation récursive suivante:
A est un sous-mot de B si:

- Cas 1: A est vide;
- Cas 2: ou A est un sous-mot de $B[1 : |B|]$;
- Cas 3: ou $A[0] == B[0]$ et $A[1 : |A|]$ est un sous-mot de $B[1 : |B|]$

Question 5;

Listing 1: *is_sous-mot*

```
def is_sous-mot(A : Array[int], B: Array[int]) -> bool:
    i = 0 # position dans A
    j = 0 # position dans C
    while(i<len(A)) and (j<len(C)):
        if A[i]==C[j]:
            i += 1
        j += 1
    return len(A)==i
```

Evaluation des complexités:

- En espace: Manipulation de variables entières uniquement $\implies \theta(n)$
- En temps: On avance de 1 à chaque itération $\implies$ au maximum $|C|$ itérations $\implies \theta(n)$

Question 6;

Listing 2: *tousSousmots*

```
def tousSousmots(B: Array[int])-> List(Array[int]):
    L = [[]]
    for x in B:
        n = len(L)
        for i in range(n):
            nouveau = L[i] + i
            L.append(nouveau)
    return L
```

Evaluation des complexités:

- En espace: On crée L et on stock toutes les entrées en fonction de leur taille n. $\implies \theta(n2^n)$
- En temps: de même, on copie tous les éléments de L[i]. Le coût dépend donc de la longueur de ce sous-mot $\implies \theta(n2^n)$

Question 7;

Listing 3: *maxLG*

```
def maxLG(B : Array[int], C: Array[int]) -> int:
    L = tousSousmots(B)
    current_max = 0
    for x in L:
        t = len(x)
        if t > current_max and is_sous-mot(x, C):
            current_max = t
    return current_max
```

- Complexité en espace: $\theta(n2^n)$ en lien avec la fonction *tousSousmots*.
- Complexité en temps: $\theta(n2^n)$ pour B et $\theta(m2^n)$ pour C $\implies \theta((n+m)2^n)$

Question 8;

On obtient, en lien avec la définiton de l'énoncé, les résultats suivants:

- $f(7,8) = 5$
- $f(0,8) = 0$
- $f(5,5) = 3$
- $f(5,4) = 3$
- $f(4,5) = 3$
- $f(4,4) = 2$
- $f(5,3) = 3$

Question 9;

$$\forall i \in [0, |B|] \quad \forall j \in [0, |C|], f(i,j) = \begin{cases} 0 & \text{si i} = 0 \text{ ou j} = 0 \\ 1 + f(i-1, j-1) & \text{si B[i-1]} = \text{C[j-1]} \\ \max(f(i-1,j), f(i,j-1)) & \text{sinon} \end{cases}$$

Question 10;

Listing 4: *maxLG*

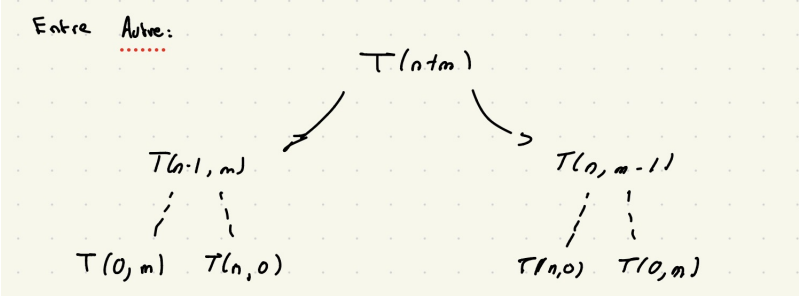
```
def maxLG(B: Array[int], C:Array[int]) -> int:
    i = len(B)
    j = len(C)
    if (i==0) or (j==0):
        return 0
    if B[i-1] == C[j-1]:
        return 1 + maxLG(B[:i-1], C[:j-1])
    return max(maxLG(B[:i-1], C[:j]), maxLG(B[:i], C[:j-1]))
```

Question 11;

Evaluation de la complexité pour l'algorithme de la question 10.
Dans le pire des cas:

- $n = |B|, m = |C|$
- $\#f(n,m) = \theta(1) + \#f(n-1,m) + \#f(n,m-1)$

Entres autres, en dessinant l'arbre des appels:



Durant chaque appel;

- peut produire au maximum 2 appels;
- la somme des longueurs n+m diminue de 1 à chaque descente dans l'arbre;
- la profondeur est au maximum de n+m

C'est donc une somme géométrique de raison 2:

$$\sum_{k=0}^{n+m} 2^k = 2^{n+m+1} - 1$$

Par conséquent:

$$\text{Complexité en temps: } \implies O(2^{n+m})$$

Pour la comlexité en espace:

- on s'intéresse ici au maximum sur la pile d'appel;
- $\implies O(n+m)$

Question 12;

Malgré l'amélioration en (Q10), on peut obtenir une situation où l'on calcul plusieurs fois un même calcul.
Pour cela: programmation dynamique.

Listing 5: *maxLG* programmation dynamique

```
def maxLG(B: Array[int], C: Array[int], memo: Array[int]) ->:
    i = len(B)
    j = len(C)
    if (i==0) or (j==0):
        return 0
    if (i,j) in memo:
        return memo[(i,j)]
    if B[i-1] == C[j-1]:
        resultat = 1 + maxLG(B[:i-1], C[:j-1], memo)
    resultat = max(maxLG(B[:i-1], C, memo), maxLG(B, C[:j-1], memo))
    memo[(i,j)] = resultat
    return memo[(i,j)]
```

Evaluation de la complexité:

- $n = |B|; m = |C|$
- En temps: $O(nm)$
- En espace: $O(nm)$

Question 13;

A est un sous-mot maximal de B et C si:

- $|A|$ est nul et $|B|$ ou $|C|$ est nul;
- B[|B|-1] = C[|C|-1] = A[|A|-1] et A[0:|A|-1] est un sous-mot maximal de B[0:|B|-1] et C[0:|C|-1]
- B[|B|-1] ! = C[|C|-1] et:
  Si $f(|B|-1, |C|) > f(|B|, |C|-1)$ alors A est un sous-mot maximal de B[0; |B|-1] et C
  Sinon : A est un sous-mot maximal de B et C[0; |C|-1]

Question 15;

Listing 6: *SousmotMaximal*

```
def SousmotMaximal(B: Array[int], C: Array[int], memo: Array[int]) -> A:List[Array[int
    ]]:
    i = len(B)
    j = len(C)
    A = []

    while i > 0 and j > 0:
        if B[i-1] == C[j-1]:
            A.append(B[i-1])
            i-=1
            j-=1
        elif memo[i-1][j] > memo[i][j-1]:
            i-=1
        else
            j-=1
    A.reverse()
    return A
```

Question 16;

Evaluation de la complexité de l'algorithme de la question ($Q15$).

- En espace: $O(nm)$
- En temps: $O(nm)$

Exercice 2:

Question 1;

– Ensemble des combinaisons de 10 et $[2, 3, 5]$:

$$[2, 2, 0]; [0, 0, 2]; [5, 0, 0]; [1, 1, 1]$$

Question 2;